



PHASE 4 • BUILD

Group Project Kickoff.

Today the learning stops and the building begins. Teams formed, leaders chosen, projects picked. Seven days of building. One demo. Let's go.

Zlaark

SESSION CARD

PROGRAM **Python Internship**

LED BY **Zlaark**

SESSION **16 of 25**

HINDU COLLEGE AMRITSAR • BCA III

12 AUGUST 2026

WEDNESDAY • BUILD WEEK BEGINS

How we'll spend our 90 minutes today.

Today is about setup and decisions — teams, projects, repos, plans. By the end, every team has a name, a leader, a project, and a plan.

00:00 – 00:10

01

🔄 Phase 3 Recap

Quick reminder of what you learned. The toolkit you'll use.

00:10 – 00:30

02

👥 Team Formation

Leaders volunteer. Leaders pick one person. I assign 1-2 to balance. Teams name themselves.

00:30 – 00:50

03

📋 Project Selection

20 projects across 3 tracks. Each team picks one. Match your ambition.

00:50 – 01:00

04

☁️ GitHub Setup

One repo per team. Everyone added. First commit today.

01:00 – 01:20

05

☰ Project Planning

Lock the scope. Write the plan. Set sprint goals. Team leader drives.

01:20 – 01:30

06

🚀 Sprint Preview + Close

What happens tomorrow (Sprint 1). Team huddle.

→ Flow: recap → form → pick → setup → plan → ship

👥 Result: a team that's ready to build tomorrow

Seven days. Five sprints. One demo.

Each sprint has a goal and a checkpoint. Your team leader drives the sprint. I move between teams every session.

📅 Today is Day 16



After Day 22: Day 23 dress rehearsal, Day 24 polish buffer, Day 25 Demo Day.

THE RHYTHM Build (1-5) → Ship (Deploy) → Rehearse (Dress) → Present (Demo)

Same rhythm every software team uses.

Leaders volunteer. Leaders pick. I balance.

I don't assign teams randomly. Here's the process — it gives the leader agency while I ensure every team is balanced.

🕒 15 minutes total

👤 THE FORMATION PROCESS

- 1 I ask: who can lead a team?** 5 min
Students who feel ready to lead raise their hands. If I don't get enough volunteers, I'll ask specific students I think are ready. Being a leader doesn't mean being the best coder — it means being organized, communicative, and responsible.
- 2 Each leader picks ONE person** 5 min
Leaders choose one person they want to work with. This gives the leader a trusted partner from the start. The picked person can't refuse (but they can tell me privately if there's a problem).
- 3 I assign 1-2 more to balance** 5 min
I add 1-2 more students to each team based on what I've seen over 15 days — coding ability, communication, attendance. Every team gets a mix of strong and growing students. No team is all-strong or all-struggling.

RESULT — Teams of 3-5. Every team has a leader, the leader's chosen partner, and 1-2 balanced additions. No one is left out. No one chooses their whole team (that creates cliques).

WHAT THE LEADER DOES

📌 TEAM LEADER RESPONSIBILITIES

- ✓ **Drives the project**
Makes sure the team hits each sprint checkpoint. Not by doing everything — by coordinating.
- ✓ **Organizes the work**
Splits features among team members. Decides who does what each sprint.
- ✓ **Communicates with me**
If the team is stuck, behind schedule, or has a conflict — the leader tells me. Not silently suffering.
- ✓ **Ensures everyone codes**
No free riders. If someone isn't contributing, the leader addresses it first, then tells me if it continues.
- ✓ **Presents on Demo Day**
The leader presents first, then each member presents their part. The leader doesn't present everything.

THE LEADER IS NOT THE BOSS — The leader drives, but decisions are discussed. The leader has the final call, but should listen. Being a leader means serving the team, not commanding it.

IF YOU WANT TO LEAD Tell me now. Being a leader looks great on a resume and in interviews. *"I led a team of 4 to build a working app."*

That's real leadership experience.

Name your team. Set up your channel. Decide your leader.

A team name creates identity. A communication channel keeps everyone connected. A confirmed leader keeps everyone accountable.

🕒 10 min • in your teams



01

TASK ONE

Name your team

- ✓ Pick a team name. Something memorable.
- ✓ Examples: “Code Warriors”, “Python Pioneers”, “Debug Squad”, “Stack Overflow”
- ✓ Use the name in your GitHub repo name: `group-N-teamname`
- ✓ This is your identity for the next 10 days. Make it something you’re proud of.



02

TASK TWO

Set up communication

- ✓ Create a WhatsApp group (or Discord, your choice)
- ✓ Add all team members + me (my number: `[INSTRUCTOR NUMBER]`)
- ✓ Group name: `“[Team Name] – Zlaark Internship”`
- ✓ This is how you coordinate: “I’m pushing the form code now.” “Can someone review my SQL?”
- ✓ No silent teammates. If someone goes quiet for 2 days, the leader checks in.



03

TASK THREE

Confirm your leader

- ✓ Once teams are formed, the team confirms who the leader is.
- ✓ If there’s disagreement, **I decide**.
- ✓ The leader’s name goes in the README: `“Team Leader: [Name]”`
- ✓ The leader can step down (tell me privately) — but only in the first 2 days. After Day 18, the leader stays.

ACCOUNTABILITY

A team name + a leader + a channel = **accountability**. No one can say “I didn’t know what was happening.” The leader coordinates, the channel communicates, the name creates identity.

15 projects. Every one uses AI. Every one is something you'd use.

Track A uses TF-IDF + scikit-learn (no external API — works offline). Track B calls free AI APIs (Groq recommended — 14,400 requests/day free). Both are demo-worthy.

15 of 25 projects

TRACK A • FOUNDATION

AI WITH WHAT YOU KNOW

- 1

Smart Quiz Generator
Paste your notes, **TF-IDF** finds the most important sentences, auto-generates 5 MCQ questions from them. Exam prep tool every student needs.
- 2

Document Similarity Finder
Upload 2+ documents, get a similarity matrix showing which docs are most alike. Like a mini plagiarism checker. **TF-IDF + cosine**.
- 3

Campus FAQ Chatbot
Type a question about college, get the closest stored answer. TF-IDF matching, chat-style UI. ChatGPT-style — but powered by **math**.
- 4

AI Resume Keyword Extractor
Upload a resume PDF, extract top 15 keywords by TF-IDF weight, show as a visual keyword cloud. Compare to a job description. **Day 14 app, refined**.
- 5

Sentiment Analyzer for Reviews
Paste product reviews, classify each as positive/negative/neutral using **scikit-learn** Naive Bayes. Show a sentiment breakdown chart. **I provide the dataset**.
- 6

Content-Based Movie Recommender
Pick a movie, get 5 similar recommendations based on genre + description TF-IDF. *"You liked Inception? Try these."* Real recommendation system.
- 7

Smart Notes Summarizer
Paste long lecture notes, get the top 5 most important sentences (by TF-IDF), plus key terms and reading time. Like a **mini TL;DR for lectures**.
- 8

Language Detector
Paste text, detect if it's English, Hindi, or Punjabi using character frequency analysis with **scikit-learn**. Useful for a multilingual campus.

Uses: TF-IDF, scikit-learn, PyPDF2, Streamlit (Days 4–14). No external AI API needed — works offline.

TRACK B • INTERMEDIATE

AI WITH FREE APIS

- 9

AI Study Buddy
Paste a topic, **Groq** generates: a summary, 3 key points, and 3 practice questions. Like having a personal tutor. Free Groq API (14,400 req/day).
- 10

Resume Tailor AI
Upload resume + paste a job description. Groq suggests 3 bullet-point improvements to make your resume match the job better. **Real career tool**.
- 11

Multi-Language Translator
Paste text, select language (Hindi, Punjabi, French, Spanish), Groq translates. Shows original + translation side by side. **Real utility on campus**.
- 12

AI Email Drafter
Select type (leave application, internship inquiry, professor follow-up), fill details, Groq generates a formal email. **Every student writes these**.
- 13

Smart Document Q&A (RAG)
Upload a PDF, ask questions about it. TF-IDF finds the relevant paragraph, Groq generates a natural-language answer. **The RAG pattern** — same tech ChatGPT uses.
- 14

AI Code Explainer
Paste Python code, Groq explains what it does line by line in simple English. Great for learning. Like having a **senior dev** explain your code.
- 15

Action Item Extractor
Paste meeting/class notes, Groq extracts: action items, deadlines, and key decisions. Structured output from unstructured text. **Like Otter.ai**.

Adds: Groq API (free, 14,400 req/day, no credit card). Same `requests.get()` pattern from Day 12 — just a different endpoint.

10 advanced projects. Multi-API. Real-world. Deployable.

These combine TF-IDF + scikit-learn + Groq API + real data sources. 2–3 could be deployed for actual college use. Portfolio pieces that make employers say: “A student built this?”

🏆 Track C • 10 projects

C TRACK C • ADVANCED REAL-WORLD AI

🏆 PORTFOLIO-GRADE • DEPLOYABLE

— PROJECTS 16 – 20

- 16** 🗣️ **AI Interview Coach**
Upload resume + paste job description. **Groq** generates 5 likely interview questions based on the job. You answer. Groq scores your answers (relevance, clarity). **Full interview prep loop.**
- 17** 📰 **AI-Powered News Digest**
Fetch news from free RSS feeds, Groq summarizes each article in 2 sentences, TF-IDF categorizes by topic, show a personalized feed. Like **In Shorts — but you built it.**
- 18** 📋 **Smart Assignment Checker**
Upload an assignment + rubric. TF-IDF checks content coverage. Groq checks grammar + gives feedback. Shows a score + improvement suggestions. **Professors would use this.**
- 19** 🔍 **AI Content Detector**
Paste text, classify as “AI-generated” or “human-written” using a trained **scikit-learn** model on a labeled dataset. The tool professors actually want. **I provide the dataset.**
- 20** 🎓 **Personal AI Tutor**
Pick a subject + topic, Groq generates a mini-lesson (explanation + example + quiz). Tracks your progress in **SQLite**. Like **Duolingo** for college subjects.

— PROJECTS 21 – 25

- 21** 📄 **AI Resume Builder + ATS Score**
Fill in details (Streamlit form), generate a formatted resume, then score it against a job description (TF-IDF + Groq suggestions for missing keywords). **End-to-end career tool.**
- 22** 🗣️ **Smart Chatbot with Memory**
A chatbot that remembers conversation history (**session_state** + SQLite), uses Groq for responses, answers follow-up questions. Like a **simplified ChatGPT with persistence.**
- 23** 📅 **AI-Powered Event Recommender**
Fetch events from free API (Eventbrite), TF-IDF matches events to user interests, Groq generates a personalized recommendation. “You like tech? Try this hackathon.”
- 24** 📄 **Document Intelligence Dashboard**
Upload any PDF/Word, get: summary (Groq), key terms (TF-IDF), sentiment (**scikit-learn**), translation (Groq), reading time, difficulty score. **All-in-one analysis dashboard.**
- 25** 📊 **AI Feedback Analyzer for College**
Collect feedback (rating + text), analyze sentiment per response (scikit-learn), generate an overall summary report (Groq), show trends on a dashboard. **Could be used for course/professor feedback.**

REAL-WORLD IMPACT

18 (Assignment Checker), **19** (AI Detector), and **25** (Feedback Analyzer) solve real problems professors have. **20** (AI Tutor) and **22** (Chatbot with Memory) are genuine AI products. These aren't demos — they're tools.

20 minutes. Read. Vote. Discuss. Decide.

The team leader drives this process. Everyone has a say. The leader makes the final call.

 Leader-driven • 20 min

THE 4-STEP PROCESS

How a team picks their project

- 1 Read the list** 3 min
Everyone reads all 20 project descriptions. No discussion yet.
- 2 Vote individually** 2 min
Each member writes their top 3 on paper. No groupthink.
- 3 Discuss** 10 min
Share your top 3. Find overlap. The leader facilitates.
- 4 Decide + tell me** 5 min
Leader locks the choice. Tells me. I approve the scope.

If you can't agree — the leader decides. That's why they're the leader.

HOW TO CHOOSE

Five questions every team should ask

- What excites the team?**
Excitement drives quality. Pick something you'd actually use.
- Can you build the core in 3 sprints?**
If the core takes more than Sprints 1-3, scale down.
- Does everyone understand it?**
If half the team doesn't get it, pick something else.
- Can you demo it in 2 minutes?**
Imagine Demo Day. 2 minutes. Is it clear?
- Are you reaching?**
Track C is ambitious. If your team is ready, go for it. If unsure, Track A or B is fine.

SCOPE DISCIPLINE A working app with **3 features** beats a broken app with **7**. Pick a project you can FINISH. You can always add features in Sprint 5 if you're ahead.

One repo per team. Everyone commits. Today.

The team leader creates the repo. Everyone else is added as a collaborator. First commit today.



github_setup.sh

BASH

```
1 # Step 1: LEADER creates the repo
2 # github.com → New repository
3 # Name: group-N-teamname (e.g. group-3-code-warriors)
4 # Set to PUBLIC. Tick "Add a README".
5
6 # Step 2: Add collaborators
7 # Settings → Collaborators → Add people
8 # Add every team member by GitHub username
9 # They get an email — they MUST accept
10
11 # Step 3: Everyone clones
12 git clone https://github.com/LEADERNAME/group-N-teamname.git
13 cd group-N-teamname
14
15 # Step 4: Add .gitignore (see right)
16
17 # Step 5: First commit
18 git add .gitignore README.md
19 git commit -m "Initial commit — [Team Name] project kickoff"
20 git push origin main
21
22 # Step 6: Everyone tests
23 # Each member clones, makes a test commit, pushes.
```

.GITIGNORE + GIT RULES

.gitignore

```
__pycache__/  
*.pyc  
.venv/  
.DS_Store  
*.db  
*.sqlite3  
.env  
data/
```

- ✓ **Commit often** — every feature, every fix
- ✓ **Meaningful messages:** "Add expense form" not "update"
- ✓ **Pull before you push** — avoid conflicts
- ✓ **Never commit** .venv or .db files
- ✓ **If merge conflict** — call me. We fix together.
- ✓ **Communicate before pushing** — tell the team

DAY 3 REVISITED

Everything from Day 3 applies now. The difference: you're in a **TEAM**. Communication is as important as code.

Scope it. Write it down. Stick to it.

Before any code, lock what you're building. The leader drives this. Everyone contributes ideas. The plan goes in the README.

PROJECT PLAN TEMPLATE

```
## Project: [NAME] ## Team: [TEAM NAME] ## Leader: [NAME] ## Members:
[NAMES] ### What it does (1 sentence): [e.g. "Upload a resume PDF and
compare it to a job description."] ### Core features (MUST have): 1.
[...] 2. [...] 3. [...] ### Nice-to-have (IF time permits): - [...] -
[...] ### Tech stack: - Streamlit (UI) - [SQLite/CSV] (storage) -
[scikit-learn/pandas] (logic) ### Sprint 1 goal: ["App runs, has
title + navigation"]
```

SPRINT BREAKDOWN

SPRINT	GOAL	CHECKPOINT
1: Skeleton	App runs, navigation	Running Streamlit app
2: Core Logic	Main feature works	Feature works on one example
3: Data	Save/load persists	Data survives restart
4: Polish	Handles bad input	Can't crash it
5: Freeze	Full run-through	Demo-ready

SCOPE RULE — Core features are MUST. Nice-to-haves are BONUS. Behind on Day 20? **DROP** the nice-to-haves. Ship the core.

WRITE IT IN THE README

Don't keep the plan in your head. Write it in README.md. Everyone can read it. Everyone knows what you're building.

Five criteria. Transparent from Day 1.

You know exactly what I'm looking for. No surprises on Demo Day.

CRITERION 01

WORKING APP 30%

Does the app work? Can you demo it start to finish without crashing?

MEASURED BY: live demo on Day 25.

"A working app with 3 features beats a broken app with 7."

CRITERION 02

CODE QUALITY 20%

Readable code? Named functions? Comments? No 200-line functions?

MEASURED BY: GitHub repo review.

"I read your code. Not just the demo."

CRITERION 03

GIT DISCIPLINE 15%

Regular commits with meaningful messages. Not one big upload.

MEASURED BY: commit history on GitHub.

"I look at your commit graph. Green squares tell the story."

CRITERION 04

SCOPE MATCH 15%

Does the finished app match the scope you locked today?

MEASURED BY: your plan vs your demo.

"Planned 3, shipped 3 = full marks. Planned 5, shipped 2 = scope mismatch."

CRITERION 05

PRESENTATION 20%

Can every student explain their part? Answer questions?

MEASURED BY: Demo Day Q&A.

"Every student presents. Every student answers at least one question."

THE BIGGEST MISTAKE Scope creep. Plan 7 features, build 3, demo broken. Lock scope today. Ship the core.

Five rules. Five anti-patterns. Read both.

Most projects fail because of people problems, not code. Here's how to avoid the traps.

✓ THE 5 RULES

- ✓ **Communicate before you push**
Tell the team: "I'm pushing now." No surprise merges.
- ✓ **Commit often, commit small**
One feature per commit. Meaningful messages.
- ✓ **Ask for help early**
Stuck 30 min? Ask the team. Stuck 1 hour? Ask me.
- ✓ **Everyone codes, everyone presents**
No one just watches. No one does everything.
- ✓ **Be honest about gaps**
"I don't understand the SQL part" is better than pretending and breaking it.

⚠ THE 5 ANTI-PATTERNS

- ✗ **The hero coder**
One person does everything. Others can't explain it on Demo Day.
- ✗ **The ghost**
Someone never shows up. Tell me privately — I'll handle it.
- ✗ **The scope creeper**
"Let's add authentication! And a chat! And..." No. Lock scope. Ship core.
- ✗ **The last-minute push**
All commits on Day 21 at midnight. I notice. Git discipline = 15%.
- ✗ **The silent conflict**
Two people disagree. Nobody talks. Both push. Merge conflict. Drama. Communicate FIRST.

THE GOLDEN RULE If something is wrong — dynamics, scope, a teammate not contributing — tell me EARLY. Day 17 is fixable. Day 24 is not.

Everything you need. You already learned it.

15 days of skills. Here's what each one is FOR — so you know what to use when.

DAYS 1-2 Python Functions, files, conditionals. The foundation.	01	DAYS 4-5 Streamlit The UI. <code>file_uploader</code>, <code>text_area</code>, <code>metric</code>, <code>columns</code>, <code>sidebar</code>.	02	DAY 6 pandas DataFrames. Load CSV, filter, sort, groupby.	03
DAY 7 Charts <code>st.bar_chart</code>, <code>st.line_chart</code>, Matplotlib.	04	DAYS 9-10 SQLite <code>CREATE TABLE</code>, <code>INSERT</code>, <code>SELECT</code>. The helper-function pattern.	05	DAY 11 Regex <code>re.findall</code>, <code>re.sub</code>. Extract patterns, clean text.	06
DAY 11 PDF/Word PyPDF2, <code>python-docx</code>. Extract text from uploaded files.	07	DAY 12 APIs <code>requests.get</code>, JSON. Live data from the internet.	08	DAYS 13-14 TF-IDF + AI <code>scikit-learn</code>. Text similarity, match scores.	09

CHEATSHEETS

All cheatsheets from Days 6-13 are still in your notes. Bookmark them. You don't need to memorize — you need to know WHERE to look.

Tomorrow: Sprint 1 — Skeleton.

Get something running. Even if it does nothing useful yet. A Streamlit app with your project's title, navigation, and placeholder screens.

WHAT TO BUILD

- ✓ `st.title("[Your Project Name]")`
- ✓ Navigation — tabs or sidebar for multiple views
- ✓ Placeholder screens — `st.write("[Feature] goes here")`
- ✓ A sidebar with team name + members
- ✓ Run it — `streamlit run app.py`

DON'T BUILD YET — No database, no logic, no API. Just the **SKELETON**. Fill it in Sprints 2-5.

CHECKPOINT

By end of Day 17: **6 ITEMS**

- ✓ A running Streamlit app
- ✓ Project title on the page
- ✓ Navigation between views (if applicable)
- ✓ Placeholder text on each view
- ✓ At least ONE commit from EACH member
- ✓ Everyone has cloned + can run the repo

IF DONE EARLY — Start Sprint 2 (Core Logic). Don't wait for me.

TOMORROW'S MINDSET

"Something that runs" beats "something perfect that doesn't." Ship the skeleton. Make it ugly. Make it work.

Today's checklist. Do all of these before you leave.

If every team completes this checklist, Day 16 is a success.

01



Team formed

You know your team. 3-5 members.

02



Team named

You picked a team name.

03



Leader confirmed

The team leader is decided and named in the README.

04



Communication channel set up

WhatsApp/Discord group with all members + instructor.

05



Project picked

One of the 20 projects (or your own, approved).

06



Scope locked

Core features + nice-to-haves written in README.

07



GitHub repo created

Leader created it. Name: `group-N-teamname`.

08



Everyone added as collaborator

All members can push. Tested with a commit.

09



`.gitignore` added

No `.venv`, no `.db` files.

10



First commit pushed

README with project name + team + plan.

IF YOU'RE STUCK

raise your hand. I'll come to your team. Fix it now, not on Day 17.

Seven days of learning. Now you use it.

Quick reminder of what you learned in Phase 3. This is the toolkit for your project.

DAY	DAY	DAY	DAY	DAY	DAY	DAY
9	10	11	12	13	14	15
Saving Data Part 1 JSON, databases, SQL, SELECT, CREATE TABLE	Building with SQLite SQLite CRUD, Streamlit pattern, expense tracker	Working With Text Regex, PDF extraction, Word docs	Calling Real APIs requests, JSON, caching, weather dashboard	A First Taste of AI TF-IDF, cosine similarity, resume matcher	Resume Matcher Build Full app: PDF + TF- IDF + Streamlit	Recap & Doubt Clearing Identify, fix, verify. Ready to build.



The learning is done. The building begins.

15 days of skills. 10 days of building. 1 demo. You have everything you need. Now go build something real.

YOUR MISSION

THE NEXT 7 DAYS

- ✓ **Day 16** — Today: teams, projects, repos
- ✓ **Day 17** — Sprint 1: Skeleton
- ✓ **Day 18** — Sprint 2: Core Logic
- ✓ **Day 19** — Sprint 3: Data
- ✓ **Day 20** — Sprint 4: Polish
- ✓ **Day 21** — Sprint 5: Freeze
- ✓ **Day 22** — Deploy (live!)

YOUR SUPPORT

WHO HELPS YOU

-  **Email** · hello@zlaark.com
-  **In-session** · raise a hand
-  **Your team** — ask them first
-  **Notes + cheatsheets**
-  **Your team leader** — coordinates

You are not alone. Ask early, ask often.

DEMO DAY

DAY 25

-  **Live demo** to faculty panel
-  Every student answers questions
-  **Certificates** for those who qualify
-  You built a real app. **From scratch.**

9 days from now, you'll show this to people who matter.